

ChequeWriter

build log

 Plain-English cheque amount converter — numeric in, NZ/British wording out.

1234.56

→

One thousand, two hundred and thirty-four dollars and fifty-six cents

Part 1 · Creating the application

Step 1 Choose .NET

Used the .NET 9 SDK with C#. The solution has two projects:

Project	Purpose
ChequeWriter	Console application (conversion logic + UI)
ChequeWriter.Tests	Automated xUnit tests kept separate from the app

Step 1: Choose .NET / C#

```
PS> dotnet --version
9.0.203

# Platform: .NET 9 SDK
# Language: C#
# Solution projects:
#   - ChequeWriter      (application)
#   - ChequeWriter.Tests (automated tests)
```

 Step 1 — .NET SDK & solution

01-choose-dotnet.png

Step 2 Create a console application

Created the solution, console app, and test project:

```
dotnet new sln -n ChequeWriter
dotnet new console -n ChequeWriter -f net9.0
dotnet new xunit -n ChequeWriter.Tests -f net9.0
dotnet sln add ChequeWriter/ChequeWriter.csproj
dotnet sln add ChequeWriter.Tests/ChequeWriter.Tests.csproj
dotnet add ChequeWriter.Tests reference ChequeWriter
```

When run, the program prompts:

Enter a cheque amount:

Step 2: Create console application + test project

```
PS> dotnet new sln -n ChequeWriter
The template "Solution File" was created successfully.

PS> dotnet new console -n ChequeWriter -f net9.0
The template "Console App" was created successfully.

PS> dotnet new xunit -n ChequeWriter.Tests -f net9.0
The template "xUnit Test Project" was created successfully.

PS> dotnet sln add ChequeWriter/ChequeWriter.csproj
PS> dotnet sln add ChequeWriter.Tests/ChequeWriter.Tests.csproj
PS> dotnet add ChequeWriter.Tests reference ChequeWriter
Reference added successfully.
```



Step 2 — console app running

02-create-console-app.png

Part 2 · Reading and validating the amount

Steps 3–4 Ask for input and validate it is a number

`Program.cs` displays the prompt with `Console.Write`, reads text with `Console.ReadLine()`, then uses `decimal.TryParse` with `CultureInfo.InvariantCulture`.

- Valid: `1234.56`
- Invalid: `twelve dollars` → clear error, exit code 1

`decimal` is used because financial amounts need decimal precision (not binary floating-point).

Steps 3-4: Ask for amount and validate numeric input

```
PS> dotnet run --project ChequeWriter
Enter a cheque amount: 1234.56
One thousand, two hundred and thirty-four dollars and fifty-six cents

PS> dotnet run --project ChequeWriter
Enter a cheque amount: twelve dollars
Invalid amount. Enter a value such as 1234.56.
Exit code: 1
```



Steps 3–4 — input & validation

03-04-ask-and-validate.png

Part 3 · Preparing the amount for conversion

Steps 5–7 Negatives, rounding, maximum

Handled inside `ChequeAmountConverter.Convert`:

1. **Reject negatives** — throws `ArgumentOutOfRangeException`
2. **Round to nearest cent** — `decimal.Round(..., 2, MidpointRounding.AwayFromZero)`
3. **Enforce maximum** — `MaxAmount = 999_999_999_999.99m`

Input	Result
-25.00	Rejected
12.345	Rounded to 12.35 → <i>Twelve dollars and thirty-five cents</i>
above max	Rejected with maximum message

Steps 5-7: Reject negatives, round to cents, enforce maximum

```
# Negative amount rejected
Enter a cheque amount: -25.00
Cheque amounts cannot be negative.
Exit code: 1

# Midpoint rounding AwayFromZero: 12.345 -> 12.35
Enter a cheque amount: 12.345
Twelve dollars and thirty-five cents

# MaxAmount = 999,999,999,999.99 (rejected when exceeded)
```



Steps 5-7 — prepare amount

05-07-prepare-amount.png

Parts 4-7 · Conversion pipeline (Steps 8-18)

Implemented in `ChequeWriter/ChequeAmountConverter.cs`:

Step	Responsibility
8-9	Split dollars (<code>Truncate</code>) and cents (<code>(amount - dollars) * 100</code>)
10-11	<code>Ones[]</code> (0-19) and <code>Tens[]</code> (twenty-ninety) word tables
12	<code>ConvertUnderOneThousand</code> — hundreds + British/NZ and
13-15	Scale groups (trillion→units), skip empty groups, commas / final and
16-18	Convert dollars & cents, singular/plural units, assemble + capitalise

Steps 8-18: Split dollars/cents, wording, singular/plural, capitalise

Enter a cheque amount: 1234.56
One thousand, two hundred and thirty-four dollars and fifty-six cents

Enter a cheque amount: 1.01
One dollar and one cent

Enter a cheque amount: 0
Zero dollars and zero cents

Enter a cheque amount: 1000001.99
One million and one dollars and ninety-nine cents

Enter a cheque amount: 21.00
Twenty-one dollars and zero cents



Steps 8-18 — conversion demo

08-18-conversion-demo.png

Method breakdown (Part 8)

Convert

Validate, round, split, label, capitalise

ConvertInteger

Whole numbers via named scales

ConvertUnderOneThousand

One group from 1-999

JoinGroups

Commas and placement of
and

CapitaliseFirstLetter

Sentence capitalisation

Part 9 · Testing (Step 19)

Ran:

```
dotnet test
```

Result: 15/15 passed



19-automated-tests.png

```
Passed Convert_ReturnsExpectedKording(1234.56)
```



19b-test-cases-detail.png

#	Input	Expected / result
1	1234.56	One thousand, two hundred and thirty-four dollars and fifty-six cents
2	0	Zero dollars and zero cents
3	1.01	One dollar and one cent
4	0.05	Zero dollars and five cents
5	21.00	Twenty-one dollars and zero cents
6	101.00	One hundred and one dollars and zero cents
7	1001.00	One thousand and one dollars and zero cents
8	1000001.99	One million and one dollars and ninety-nine cents
9	12.344	Twelve dollars and thirty-four cents
10	12.345	Twelve dollars and thirty-five cents
11	12.999	Thirteen dollars and zero cents
12	-1.00	Rejected

#	Input	Expected / result
13	twelve dollars	Invalid amount message (console)
14	9999999999999999.99	Full scale wording accepted
15	above max / 1234567	Rejected / large grouped wording

How to run

```
# Application
dotnet run --project ChequeWriter

# Tests
dotnet test
```

⚙️ Production note (as in the brief)

Before production use, confirm with the business:

1. Whether fractions of a cent should be **rounded** (current behaviour) or **rejected**
2. Whether wording should end with **only** (e.g. ... *cents only*)